

DTL: Random Forest

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Vom einzelnen Baum zum Wald

- Idee:
 - Statt **einem** Baum viele **verschiedene** Bäume trainieren
 - Jeder Baum trifft eine eigene Vorhersage
 - Am Ende: **Mehrheitsentscheidung** bzw. **Mittelwert**
- Intuition:
 - “Viele verschiedene Meinungen” kombinieren → robustere Entscheidung
 - Fehler einzelner Bäume können sich gegenseitig ausgleichen

Wie funktioniert ein Random Forest?

1. **Datenbasis:** Trainingsdatensatz D
2. **Viele (Teil-) Trainingsdatensätze durch Zufall (Bootstrapping/Bagging)**
 - Für jeden Baum $B_1, B_2, \dots, B_k$:
 - Zufälliges Ziehen (mit Zurücklegen) der Trainingsbeispiele aus D
→ jeder Baum erhält eine leicht andere Stichprobe D_k
3. **Zufällige Merkmalsauswahl pro Split (Feature-Subsampling)**
 - Beim Training eines Baums:
 - an jedem Knoten nur zufällige Teilmenge der Attribute für den Split betrachten
 - daraus das “beste” Attribut wählen (Entropie)
 - Alternativ: Anzahl der Splits begrenzen
4. **Vorhersage**
 - Klassifikation: Mehrheitsentscheid der Vorhersagen der Bäume
 - Regression: Mittelwert der Vorhersagen der Bäume

Beispiel: Random Forest in Aktion (Klassifikation)

Bäume:

Größe
/g \k
A B

Farbe
/d \h
A B

Größe
/g \k
Farbe B
/d \h
B A

Testobjekt: (Größe = groß, Farbe = hell)

Vorhersage:

A

B

A

Mehrheitsentscheidung: 2x A, 1x B → **A**

Pseudocode: Random Forest

Training:

```
Input: Trainingsdaten D, Anzahl Bäume K,  
       Anzahl Zufallsattribute pro Split m  
  
For k = 1 .. K:  
    Ziehe Zufallsstichprobe D_k aus D (mit Zurücklegen)    // bagging  
    Trainiere Entscheidungsbaum T_k auf D_k:  
        An jedem Knoten:  
            Wähle zufällig m Attribute aus allen Attributen  
            Finde unter diesen m Attributen den besten Split (Entropie)  
Output: Ensemble {T_1, T_2, ..., T_K}
```

Vorhersage für ein neues Objekt x :

- Klassifikation: $\hat{y}_{\text{class}}(x) = \operatorname{argmax}_{c \in C} \sum_{k=1}^K \mathbf{1}[T_k(x) = c]$
- Regression: $\hat{y}_{\text{reg}}(x) = \frac{1}{K} \sum_{k=1}^K T_k(x)$

- **Random Forest:**
 - Kombiniert viele (kleine) Entscheidungsbäume
 - Training auf zufälligen Teilmengen der Daten und Merkmalen (Bagging + Feature-Subsampling)
- Gesamtvorhersage: **Mehrheitsvotum** der Bäume bzw. Mittelwertbildung
- Ensemble **reduziert Varianz**
- Wichtige Hyperparameter:
 - Anzahl der Bäume
 - Anzahl der Merkmale pro Split
 - Maximale Baumtiefe



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

